

Registro para Minutas 1

Estudiante	Minuta 1			
	Tareas realizadas	Resultados obtenidos	Próximo paso	Fecha / Evaluación
Alejandro Cañas Sánchez	Busqueda información acerca de la programación dinámica ya que nos piden de un montón de pedidos, elegir cuáles caben en el camión para ganar el máximo dinero posible usando Programación Dinámica. Organización de tareas	Estuve investigando cómo aplicar la Programación Dinámica al problema de la mochila para la selección de los pedidos que nos piden en el enunciado y en que se diferencia del algoritmo voraz que ya hemos estudiado en clase	Comenzar con el desarrollo del código para el módulo <code>dp_seleccion.py</code> y realizar las primeras pruebas con datos	
Rubén Tostón Carrero	Creación del repositorio en Github para que todos los integrantes del grupo puedan ver y actualizar en tiempo real el proyecto, y organización de carpetas según el pdf	He creado a través de la web el repositorio de Github en el que trabajaremos. Como Github no reconoce carpetas vacías, he creado archivos temporales vacíos para que Github recuerde la estructura. Según vayamos añadiendo scripts al proyecto, esos archivos se irán borrando	Una vez creado el repositorio, se puede crear en la carpeta designada el script <code>main.py</code> para la ejecución de los scripts y de pruebas	
Miguel Huete Curvi	Creación del diseño de la matriz de adyacencia que representa el grafo de la ciudad y definición de los cinco escenarios de prueba	He creado un grafo ponderado y no dirigido a través de una matriz de adyacencia que representa la ciudad del problema. También se han creado los cinco escenarios de pruebas solicitados.	Comenzar con el diseño del algoritmo de backtracking para la ruta.	

Nota: Cada minuta se evalúa en base a 10 y luego se promedia su nota. Todos los estudiantes tienen que presentar tres avances.

Registro para Minutas 2

Estudiante	Minuta 2			
	Tareas realizadas	Resultados obtenidos	Próximo paso	Fecha / Evaluación
Alejandro Cañas Sánchez	Pensar entre todos un contexto para dar sentido al problema y sea más fácil para desarrollar mejoras y hacer el código de 'dp_seleccion.py', así como posteriores mejoras.	Primeramente, desarrollé un contexto al que aplicar nuestro algoritmo para hacer la implementación de mejoras posteriores de forma amena. Además, programé el código de selección en el script 'dp_seleccion.py', además de mejorarlo incluyendo casos con varios vehículos y beneficios. Por último, creé un grafo animado en html para poder visualizar las rutas más fácilmente	Ir pensando más mejoras que incluir al algoritmo tales como un cálculo interno de la complejidad del algoritmo completo contando las veces que recurre al cálculo de rutas.	
Rubén Tostón Carrero	Creación del script principal y el ejecutable 'main.py', así como mejora y conexión entre los otros scripts que participan en el algoritmo. A parte, creación de un script adicional 'floyd_warshall.py' como requerimiento de mejora del programa.	He creado el 'main.py' que conecta los demás archivos y lo he probado con tests que han resultado en éxito. Después de mejorar el script, pude crear el script 'floyd_warshall.py', que como su nombre indica, utiliza el algoritmo de Floyd_Warshall para explorar todos los nodos y encontrar la ruta aún más optimizada.	Pensar más funcionalidades a añadir en el algoritmo, sobre todo referentes a comparaciones con un algoritmo Voraz para ver las ventajas de la PD	
Miguel Huete Curvi	Creación del algoritmo backtracking para la ruta de los pedidos y mejora de la matriz de adyacencia del grafo de Alcalá.	A partir del grafo ponderado, he creado el script 'backtracking_ruta.py' para incluirlo en el algoritmo general. Ha sido testeado con éxito con los datos de simulación (matriz de adyacencia)	Incluir mejoras, realizar pruebas con más volumen de datos y aplicar escenarios, tales como hacer que el algoritmo extraiga los datos de archivos JSON.	

Nota: Cada minuta se evalúa en base a 10 y luego se promedia su nota. Todos los estudiantes tienen que presentar tres avances.

Registro para Minutas 3

Estudiante	Minuta 3			
	Tareas realizadas	Resultados obtenidos	Próximo paso	Fecha / Evaluación
Alejandro Cañas Sánchez	Realización de un html que sirve de interfaz grafica para entender y explicar nuestro proyecto, añadido unos multiplicadores de velocidad depende del vehiculo y estructuración memoria	He creado una interfaz interactiva con un grafo que muestra a tiempo real como funciona el programa, observábamos que una furgoneta se movia igual de rápido que un patinete arreglamos ese problema. He creado y editado la memoria y le he dado el formato adecuado	Comprobar que los cambios de velocidad funcionan, el html no tiene ningún fallo y ver que la memoria es correcta	
Rubén Tostón Carrero	Añadidas funciones que usan algoritmo voraz y PD. Añadida función que permite al repartidor repartir todos los pedidos en tandas. Rellenado del archivo README.md. Implementación y revisado en el main	He añadido una nueva función que utiliza un algoritmo voraz, ordenando los pedidos de mayor a menor beneficio bruto, para comparar diferentes casos respecto al algoritmo PD. Además, he añadido un nuevo script 'multi_viaje.py' que incluye una mejora al programa: si un vehículo no tiene actualmente espacio para llevar más pedidos, sabe repartir los pedidos que ya tiene para luego volver al almacén a abastecerse de nuevos pedidos	Revisar que todo está bien cohesionado y las llamadas entre funciones son correctas, además de añadir casos de prueba extremos aparte de los ya incluidos en el proyecto para ver cómo se comporta el programa. Luego probar el caso general realista	
Miguel Huete Curvi	Diseño, estandarización y creación de los 5 escenarios de prueba solicitados en formato JSON dentro de la nueva estructura de directorios del repositorio (data/escenarios/). Corrección de la dimensionalidad de la matriz de Alcalá para alinearla con la interfaz gráfica.	El algoritmo TSP ahora es completamente robusto frente a entradas incorrectas. Además, los escenarios JSON se han desacoplado con éxito, permitiendo que el main.py inyecte dinámicamente diferentes vehículos en los mismos mapas sin fallos de lectura, validando así la integración de los tres módulos del grupo.	Revisión general de las conclusiones en la memoria redactada para asegurar que la complejidad espacial y temporal reflejada cuadra con el código final antes de la entrega.	

Nota: Cada minuta se evalúa en base a 10 y luego se promedia su nota. Todos los estudiantes tienen que presentar tres avances.

Que es y que puede contener las minutas

Las minutas se contemplan como segunda evaluación y no son más que un reporte bisemanal (cada dos semanas) de los avances de la práctica, específicamente de las actividades y/o tareas llevadas a cabo por el grupo reflejando todas las consideraciones del caso, entre los que se puede destacar.

1.- Búsqueda de información para documentación.

2.- Investigación bibliográfica.

3.- Desarrollo de diagramas con detalles de propuestas.

4.- Pruebas realizadas de código. Demostraciones.

5.- Pruebas y resultados obtenidos de esto.

6.- Resultados obtenidos en reuniones con compañeros del mismo grupo, otros grupos, consulta a expertos, incluyendo el profesor.

7.- Otras consideraciones necesarias que permitan el avance de la práctica.